Experiments 7 Bug

1. 报错内容

```
(brokerlab) PS
D:\BaiduNetdiskDownload\exp_anvil_broker_qwen\exp_anvil_broker_qwen\experiments\exp007_dynamic_routing> python .\run.py
  exp007 · out:
D:\BaiduNetdiskDownload\exp_anvil_broker_qwen\exp_anvil_broker_qwen\experiments\exp007_dynamic_routing\out\20260922_213035 | dyn rates=[120] | static rates=[120]
  workload: 40000 笔 → 50 bins(静态臂) | 合计 118974 ETH

  ==== 静态对照 120 CTX/block ====
Traceback (most recent call last):
  File
"D:\BaiduNetdiskDownload\exp_anvil_broker_qwen\exp_anvil_broker_qwen\experiments\exp007_dynamic_routing\run.py", line 923, in <module>
    sys.exit(main())
             ^^^^^^
  File
"D:\BaiduNetdiskDownload\exp_anvil_broker_qwen\exp_anvil_broker_qwen\experiments\exp007_dynamic_routing\run.py", line 832, in main
    st = run_static(cfg, r, bins, sel, total_wei, out_root, watchdog)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"D:\BaiduNetdiskDownload\exp_anvil_broker_qwen\exp_anvil_broker_qwen\experiments\exp007_dynamic_routing\run.py", line 357, in run_static
    after = {(a, s): tx.get_balance(a, s) for (a, s) in before}
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File
"D:\BaiduNetdiskDownload\exp_anvil_broker_qwen\exp_anvil_broker_qwen\experiments\exp007_dynamic_routing\run.py", line 357, in <dictcomp>
    after = {(a, s): tx.get_balance(a, s) for (a, s) in before}
                     ^^^^^^^^^^^^^^^^^^^^
  File
"D:\BaiduNetdiskDownload\exp_anvil_broker_qwen\exp_anvil_broker_qwen\brokerlab\tx.py",
line 46, in get_balance
    return self._conns.web3(shard).eth.get_balance(address)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\hzy\.venvs\brokerlab\Lib\site-packages\web3\eth\eth.py", line 442, in
get_balance
    return self._get_balance(account, block_identifier)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\hzy\.venvs\brokerlab\Lib\site-packages\web3\module.py", line 112, in
```

```
caller
    result = w3.manager.request_blocking(
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\hzy\.venvs\brokerlab\Lib\site-packages\web3\manager.py", line 233, in
request_blocking
    return self.formatted_response(
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\hzy\.venvs\brokerlab\Lib\site-packages\web3\manager.py", line 199, in
formatted_response
    validate_rpc_response_and_raise_if_error(
  File "C:\Users\hzy\.venvs\brokerlab\Lib\site-packages\web3\_utils\validation.py", line
401, in validate_rpc_response_and_raise_if_error
    raise web3_rpc_error
web3.exceptions.Web3RPCError: {'code': -32602, 'message': 'BlockOutOfRangeError: block
height is 387 but requested was 386'}
(brokerlab) PS
D:\BaiduNetdiskDownload\exp_anvil_broker_qwen\exp_anvil_broker_qwen\experiments\exp007_d
ynamic_routing>
```

2. AI的解释（短）

不完全是。更准确地说：

1. 程序并没有“本来打算读取 386 区块”。
2. `get_balance()` 没指定区块，因此请求的是：

```
读取 latest（执行请求时的最新区块）
```

3. Anvil 开始处理请求时，`latest` 是 386，于是内部暂时记成：

```
requested block = 386
```

4. 处理尚未结束时，Anvil 自动生成了第 387 块：

```
current block = 387
```

5. Anvil 再去读取刚才记录的 386 状态时，因为启用了历史裁剪，无法正常取得，于是报错：

```
block height is 387 but requested was 386
```

完整过程是：

```
程序请求 latest
      ↓
Anvil 此刻将 latest 解析为 386
      ↓
恰好生成第 387 块
      ↓
Anvil 仍继续读取 386
      ↓
386 的历史状态不可用
      ↓
报错，没有返回任何余额数据
```

所以并不是“读取到了 387 的数据，和需要的 386 数据不一致”。实际上这次调用什么余额都没有返回，直接抛出了异常。

另外，`get_balance()` 不指定区块本身是正常写法。真正的问题是：

```
Anvil 处理 latest 请求时恰好遇到自动出块
+
启用了 --prune-history
+
程序没有对这种瞬时 RPC 错误进行重试
```

即使显式指定 386，也未必能解决，反而可能因为 386 已被裁剪而稳定失败。正确处理方式是：继续读取 `latest`，但遇到这类出块竞态时等待几十毫秒后重新查询。重试时 `latest` 会稳定解析为 387，从而成功返回余额。

3. AI的解释（长）

你说得对。下面直接按照这次运行的真实数据讲解，不使用比喻。

这次错误发生在：

```
实验：exp007_dynamic_routing
运行目录：out/20260922_213035
实验臂：stat_120
出错分片：shard_5
Anvil 端口：8605
```

1. 程序当时在执行什么？

`stat_120` 的 40,000 笔 CTX 已经处理完毕。

随后程序运行这段代码：

```
after = {
    (a, s): tx.get_balance(a, s)
    for (a, s) in before
}
```

位置在 run.py (line 357)。

它的作用是读取实验结束后每个账户在每个分片上的余额，用于检查：

```
实验前总余额 == 实验后总余额
```

假设需要检查这个账户在分片 5 的余额：

```
账户：0x4233a52ec754be0998a9020c4c406b49169ad2f1
分片：5
```

程序实际调用：

```
tx.get_balance(
    "0x4233a52ec754be0998a9020c4c406b49169ad2f1",
    5
)
```

2. Web3 实际发送了什么请求?

项目中的实现是：

```
def get_balance(self, address: str, shard: int) -> int:
    return self._conns.web3(shard).eth.get_balance(address)
```

这里没有提供 `block_identifier`，所以 web3.py 会发送类似下面的 JSON-RPC 请求：

```
{
  "jsonrpc": "2.0",
  "method": "eth_getBalance",
  "params": [
    "0x4233a52ec754be0998a9020c4c406b49169ad2f1",
    "latest"
  ],
```

```
  "id": 1
}
```

`latest` 表示读取该分片最新区块对应的账户余额。

`shard_5.log` 也记录了真实请求：

```
Request: EthGetBalance(
    0x4233a52ec754be0998a9020c4c406b49169ad2f1,
    Some(latest)
)
```

因此程序没有主动指定第 386 块。

3. 出错时区块发生了什么变化?

你的 Anvil 启动参数包含：

```
--block-time=1
```

因此每秒生成一个新区块。

发生错误时，`shard_5` 正好从第 386 块推进到第 387 块。实际执行顺序很可能是：

```
① 收到 eth_getBalance(address, "latest")
② Anvil 将 latest 解析为区块 386
③ 自动出块线程生成区块 387
④ Anvil 继续尝试读取刚才解析出的区块 386 状态
⑤ 发现节点当前高度已经是 387
⑥ 区块 386 的历史状态不可用于这次查询
⑦ 返回 BlockOutOfRangeError
```

真实日志内容是：

```
Block Number: 387
Block Hash: 0x53b86cd6a1970242085e65b93963bb801ebb9e050509d2554ce8fc0e7cf32769

RPC request failed:
    Request: EthGetBalance(
        0x4233a52ec754be0998a9020c4c406b49169ad2f1,
        Some(latest)
    )
    Error: Invalid params:
```

```
        BlockOutOfRangeError:
        block height is 387 but requested was 386
```

这里两个数字的含义是：

```
block height is 387
```

表示 Anvil 处理请求时，链已经运行到第 387 块。

```
requested was 386
```

表示这次 `latest` 查询在稍早时刻被解析成了第 386 块。

问题在于：同一个 RPC 请求开始处理时最新块是 386，继续处理时最新块已经变成了 387。

4. `--prune-history` 有什么影响?

Anvil 还使用了：

```
--prune-history
```

该参数位于 chain.py (line 117)。

开启后，Anvil不会长期保存所有旧区块的完整状态。因此当节点从 386 推进到 387 后，针对 386 的状态读取可能不可用。

如果保留完整历史状态，那么即使节点已经到 387，也可能继续读取 386 的余额状态。现在开启了历史裁剪，Anvil只能返回错误。

5. 为什么一次错误会让整个实验退出?

Python 的余额读取目前没有重试：

```
return self._conns.web3(shard).eth.get_balance(address)
```

所以调用链变成：

```
eth_getBalance
    ↓
Anvil 返回 RPC 错误
    ↓
web3.py 抛出 Web3RPCError
    ↓
```

```
get_balance 没有捕获
    ↓
run_static 没有捕获
    ↓
main 没有捕获
    ↓
整个 run.py 退出
```

这就是你看到完整 Traceback 的原因。

6. 正常重试时会怎样?

假设第一次查询发生在出块边界:

```
21:37:12.999
请求 latest
Anvil 暂时解析为 386
```

紧接着:

```
21:37:13.000
Anvil 生成第 387 块
第一次查询失败
```

程序等待 50 毫秒再次查询:

```
21:37:13.050
再次请求 latest
Anvil 解析为 387
读取第 387 块余额成功
```

第二次可能返回:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "result": "0x..."
}
```

因为所有实验交易已经完成，第 386 块和第 387 块之间通常只是产生了一个空块，所以重新读取第 387 块不会改变实验余额结果。

7. 为什么不是 `rate=120` 的业务错误?

如果是注入速率或交易处理问题，错误通常会出现在：

```
发送交易
等待交易回执
nonce 管理
backlog 检查
交易超时
```

对应错误可能是：

```
transaction underpriced
nonce too low
insufficient funds
receipt timeout
backlog > max_backlog
```

但这次错误发生在：

```
40,000 笔任务处理结束
→ 停止 coordinator
→ 读取最终余额
→ 某一次 eth_getBalance 失败
```

错误类型也是：

```
Web3RPCError: BlockOutOfRangeError
```

因此它属于 Anvil 状态读取问题，不是 `rate=120` 直接造成的交易执行错误。

实际修复应当是在 tx.py (line 44) 中，仅对包含 `BlockOutOfRangeError` 的余额查询进行几次短间隔重试。其他 RPC 错误仍应正常抛出，避免隐藏真正的实验故障。